Assignment 2: From Trees to Neural Networks

Submission

- A runnable Jupyter/Colab Notebook (`.ipynb`), with the setup, data-loading instructions, and outputs needed to reproduce your results.
- A PDF report (**5–7 pages, including figures and references**) submitted on Gradescope.
    - Include a page map showing where you address Steps 1–5.

A Note Before You Start

Use AI tools freely, and be ready to explain your own code. Why did you scale these features? What changed when you increased the learning rate? How do you know that a better score came from a better model?

Objective

Compare `gradient boosted decision trees (`GBDT`) and a multi-layer perceptron (`MLP`) on tabular data. Use controlled experiments to analyze predictive performance, learning behavior, training time, and interpretability. **A GPU is not required.**

Datasets

Choose one of the following datasets. Bank Marketing is the simpler option to download and run. Both options receive the same grading treatment.

Bank Marketing Dataset

Task: Predict whether a client subscribes to a term deposit (y = yes is the positive class).
Dataset link: UCI Bank Marketing. No account is required.

- Use `bank-additional-full.csv` (41,188 rows) from `bank-additional.zip`; the CSV separator is a semicolon.
- Drop `duration` from the predictors. It is the completed call's `duration` and is unavailable when making a prediction before that call. The dataset documentation explicitly flags this leakage risk.
- Explain how you handle unknown categories and special values such as `pdays = 999` (no previous contact).

Home Credit Default Risk Dataset

Task: Predict the positive class in `TARGET`, which identifies applicants with payment difficulties.
Dataset link: Kaggle Home Credit Default Risk. A Kaggle account and acceptance of the competition's data terms may be required.

- Use `application_train.csv` only; drop `SK_ID_CURR` and separate `TARGET` from the predictors. Additional table joins and Kaggle submissions are not required.

- For this assignment, take a **30,000-row stratified sample**, using `TARGET` for stratification and `random_state = 42`, before making your train/validation/test split. Use that same sample for every experiment.

These are historical benchmarks for studying model behavior. Do not interpret their scores as evidence of current banking performance. For Bank Marketing, note that a random split also does not test generalization to future campaigns.

Steps

1. **Data Preparation**
    a. Report the number of rows, feature types, positive-class proportion, and missing or special values. Justify your cleaning choices.
    b. Create a stratified **70/15/15** train/validation/test split with `random_state = 42`.
    c. Fit learned preprocessing **only on training data**. Use one-hot encoding for nominal categories and apply the fitted transformations to validation/test data.
    d. **Scale numeric features** for the MLP; scaling is optional for trees. Keep the same input information and split rows for both models, and explain any model-specific preprocessing.
2. **Gradient Boosted Trees (GBDT)**
    a. Train `xgboost.XGBClassifier` with `tree_method = "hist"`. Use binary log loss to monitor training and validation performance.
    b. Compare at least three `learning_rate` values (for example, 0.01, 0.1, 0.3), holding the other settings fixed. Then vary `max_depth` and one regularization control: `subsample`, `reg_alpha`, or `reg_lambda`.
    c. Set a reasonable upper limit on `n_estimators` and use validation-based early stopping. Put `early_stopping_rounds` and `eval_metric` in the `XGBClassifier` constructor; pass the transformed training and validation data through `eval_set` in `fit()`. **List validation last** so it controls early stopping.
    d. Include a training-versus-validation log-loss plot, a feature-importance plot for the selected model, and a table showing your tried settings and validation scores. State which feature-importance measure you used.
    e. Explain how learning rate, tree complexity, and regularization changed convergence or overfitting.
3. **Multi-Layer Perceptron (MLP)**
    a. Train `sklearn.neural_network.MLPClassifier` with `solver = "adam"`.
    b. Compare at least three `hidden_layer_sizes` settings (for example, `(64,)`, `(128, 64)`, and `(128, 64, 32)`), holding the other settings fixed. Then vary `learning_rate_init` and one of `activation` or `alpha` (L2 regularization).
    c. Use `max_iter` as a training budget and report convergence warnings. For a simple comparison, use `early_stopping = False`.

d. If you enable built-in early stopping instead, explain that it reserves an internal validation subset from the training data. Document that choice and continue using the shared external validation set for model selection.
e. Include the selected model's training-loss curve (`loss_curve_`) and a plot or table of validation performance versus architecture. Explain how depth/width, learning rate, and regularization affected the results.

Experiment budget: Aim for roughly **6–10 configurations per model**. The experiments above can share configurations; a full Cartesian grid is not required. Record every configuration you tried and keep the search effort comparable across the two models.

4. **Model Evaluation & Comparison**
    a. Choose one final configuration per model using **validation average precision (AP)**, the primary metric for this assignment. **Keep the test set untouched** until all choices are fixed, then evaluate those fitted models once.
    b. Report Accuracy, Precision, Recall, F1, and **Average Precision (AP)** for the positive class. Compute AP using `average_precision_score` on predicted positive-class probabilities; do not use thresholded labels or trapezoidal PR area.
    c. Use a threshold of 0.5 for Accuracy, Precision, Recall, and F1. If you also tune a threshold, choose it on validation data and label the additional results clearly.
    d. Include both models' PR curves and a side-by-side comparison table. Report the test positive-class proportion as the reference level for the PR plot, and include confusion matrices.
    e. Compare final-model fit time on the same machine/runtime and report preprocessing time separately. Record the hardware, package versions, and random seeds; do not compare a CPU run with a GPU run as if hardware were identical.
    f. Discuss when you would prefer GBDT or MLP, their sensitivity to hyperparameters and scaling, and how preprocessing handles categories and missing values. Feature importance is an association-based explanation, not a causal conclusion; MLPs can also be examined with model-agnostic methods.

5. **Report Writing**
    a. **Introduction:** Define the prediction task and describe the chosen dataset.
    b. **Methods:** Explain preprocessing, the split, model settings, and your tuning experiments.
    c. **Results:** Present metrics, figures, training time, and the GBDT-versus-MLP comparison.
    d. **Discussion:** Interpret the results, reflect on bias and variance, and identify limitations. Neither model is required to win; there is no fixed minimum F1 or AP score for full credit.
    e. **AI Tool Disclosure:** List the tools used, describe their roles, and explain what you checked or changed yourself. State explicitly if you did not use AI tools.

Pointers & Hints

- Use pandas, NumPy, scikit-learn, XGBoost, and matplotlib/seaborn.
- `ColumnTransformer` and `Pipeline` help keep preprocessing consistent.
- If you use cross-validation, fit preprocessing within each training fold and keep any early-stopping holdout separate from that fold's scoring data.
- Official references: XGBoost sklearn interface and early stopping, MLPClassifier, and average_precision_score.

Report Formatting & Visualization Rules

- Embed all required graphs and tables directly in the PDF. We will not open your notebook to find missing figures.
- Keep code out of the report, including appendices. Use readable labels and captions; do not shrink figures to fit more experiments.

Grading Rubric

Category	Percentage	Description
Data Preparation	15	Correct split and preprocessing; justified choices; no leakage.
GBDT Implementation	15	Correct training, controlled tuning, early stopping, and learning dynamics.
MLP Implementation	15	Correct scaling, training, controlled tuning, and learning dynamics.
GBDT vs. MLP Comparison	25	Fair comparison of predictive performance, time, and model behavior; thoughtful analysis.
Evaluation & Visualization	20	Correct metrics and clear required figures embedded in the PDF.
AI Tool Usage Disclosure	10	Transparent account of AI assistance and your own verification and contribution.

Colab Link & Access Requirements

- Include your Google Colab notebook link **at the very end** of your submitted PDF report.
- Set your notebook's sharing permissions so that anyone signed in with a **Columbia LionMail account** can open it as a Viewer without requesting access. Verify this using a different LionMail account before submitting.
- If we cannot open your Colab notebook using a LionMail account during grading, **10 points** will be deducted from the assignment score.